

ECE0202 Lecture 16

RTC: Real-Time Clock 中文逐页详细讲义

目标：把 RTC 时钟、BCD、初始化流程、Alarm 和 EXTI18 中断链路讲清楚。

本讲主线： RTC 是 STM32 里独立计时的日历时钟模块。它可以用低功耗时钟源持续运行，记录年月日时分秒，并在设定时间产生 Alarm interrupt。本讲不是单纯背寄存器，而是理解四条链路：**时钟源** → **1 Hz 日历时钟** → **时间/日期寄存器** → **Alarm 触发中断**。

Contents

1	Slide 1: 标题页 - Lecture 16 RTC	2
2	Slide 2: Announcement - RTC Clock 与 Alarm	2
3	Slide 3: Real-Time Clock (RTC)	2
4	Slide 4: RTC Calendar	2
5	Slides 5–6: Date & Time - RTC_TR 与 RTC_DR	3
6	Slides 7–8: BCD Format 与例子	3
7	Slide 9: Steps to Set the Time for RTC	4
8	Slide 10: Clock Source	4
9	Slides 11–13: Clock Configuration - RTC 分频得到 1 Hz	5
10	Slides 14–21: RTC Initialization (8 Steps) 总览	5
11	Slide 14: RTC Initialization Step 1 - 解锁 RTC_WPR	6
12	Slides 15–16: RTC Initialization Steps 2–3 - INIT 与 INITF	6
13	Slide 17: RTC Initialization Step 4 - 配置 RTC_PRER	6
14	Slide 18: RTC Initialization Step 5 - 写 RTC_TR 与 RTC_DR	7
15	Slide 19: RTC Initialization Step 6 - RTC_CR 的 FMT 位	7
16	Slide 20: RTC Initialization Step 7 - 退出初始化模式	8
17	Slide 21: RTC Initialization Step 8 - 重新锁定 RTC_WPR	8
18	Slides 22–23: Read From RTC - 读取 RTC 时间	8
19	Slides 24–25: Alarm 概念	8
20	Slide 26: RTC Alarms - Alarm A 与 Alarm B	9
21	Slide 27: Set Alarm - Every Monday vs Every Day	9
22	Slide 28: Steps to Configure ALARM A	10

23 Slides 29–30: Alarm Interrupts - RTC_CR 与 RTC_ISR	10
24 Slides 31–32: RTC Alarm Interrupt 连接到 EXTI18	10
25 Slides 33–42: EXTI 寄存器与 RTC Alarm 相关配置	11
26 Slide 43: RTC Alarm Interrupt Handler - 在哪里检查代码	12
27 Lecture 16 总结: 你必须掌握的内容	12

Slide 1: 标题页 - Lecture 16 RTC

这一页只是课程标题。真正内容从 Slide 2 开始。

先建立大图景：之前的 Lecture 13–15 讲的是 interrupt 机制和 EXTI。Lecture 16 把这些机制用到一个具体外设：**RTC** (Real-Time Clock, 实时时钟)。RTC 不只是“一个计数器”，它更像一块微控制器内部的电子手表。

Slide 2: Announcement - RTC Clock 与 Alarm

这一页列出本讲两个主题：

- **RTC Clock** (RTC 时钟)：RTC 靠哪个时钟源运行，怎样得到 1 Hz 的日历节拍。
- **Alarm** (闹钟)：在指定时间触发事件或中断。

你要先区分两个概念：

- **RTC 本身在计时**：比如现在是 16:50:00，下一秒变成 16:50:01。
- **Alarm 是条件触发器**：比如当时间等于 Monday 16:50:00 时，产生 alarm flag 或 interrupt。

不要把 RTC 和 Timer 混为一谈。普通 timer 更像“秒表/周期计数器”；RTC 更像“日历表”，能保存小时、分钟、秒、日期、星期、月份、年份。

Slide 3: Real-Time Clock (RTC)

RTC (Real-Time Clock, 实时时钟) 是一个数字时钟模块，能提供 **calendar time and date**，也就是日历时间和日期。

RTC 的典型要求

- **Low power consumption** (低功耗)：RTC 经常需要在系统睡眠时继续工作，所以必须省电。
- **Separately powered by a battery** (可由独立电池供电)：很多系统即使主电源断了，RTC 也要继续走时。
- **Accurate** (准确)：时间不能乱飘，因此 RTC 常用 32.768 kHz 晶振。
- **Run independently from processor core** (独立于处理器内核运行)：CPU 可以睡眠或忙别的，RTC 仍然计时。

直白理解：RTC 是芯片里一块“低功耗电子表”。CPU 不需要一直数秒，RTC 自己数。需要时 CPU 读 RTC 寄存器，就能知道当前时间。

Slide 4: RTC Calendar

这一页展示 RTC 的日历能力。RTC 不只是秒计数器，它能表达：

- **Seconds** (秒)
- **Minutes** (分钟)
- **Hours** (小时)
- **Day of week** (星期几)
- **Date** (一个月中的第几天)
- **Month** (月份)
- **Year** (年份，通常只存两位年，例如 23 表示 2023)

为什么要有 Calendar?

如果只有普通 timer, 你最多知道“过了多少秒”。但日历系统需要知道“现在是几月几日星期几几点几分几秒”。RTC 直接提供这种结构化日期时间, 适合闹钟、数据记录时间戳、低功耗唤醒。

很多初学者把 RTC 理解成“每秒加一的变量”。这是低级版本。STM32 RTC 的重点是: 它内部已经按 BCD 格式组织好了时分秒年月日星期。

Slides 5–6: Date & Time - RTC_TR 与 RTC_DR

这两页分别展示两个核心寄存器:

- RTC_TR: **RTC time register** (RTC 时间寄存器)
- RTC_DR: **RTC date register** (RTC 日期寄存器)

1. RTC_TR: 存时、分、秒

RTC_TR 主要保存:

- HT: Hour Tens, 小时十位
- HU: Hour Units, 小时个位
- MNT: Minute Tens, 分钟十位
- MNU: Minute Units, 分钟个位
- ST: Second Tens, 秒十位
- SU: Second Units, 秒个位
- PM: AM/PM 标志, 12 小时制时使用

例如时间 13:59:30, 在 BCD 思维下可以拆成:

$$13 : 59 : 30 \Rightarrow H = 13, M = 59, S = 30$$

即: 小时十位 1、小时个位 3、分钟十位 5、分钟个位 9、秒十位 3、秒个位 0。

2. RTC_DR: 存年、月、日、星期

RTC_DR 主要保存:

- YT: Year Tens, 年份十位
- YU: Year Units, 年份个位
- WDU: Week Day Units, 星期几
- MT: Month Tens, 月份十位
- MU: Month Units, 月份个位
- DT: Date Tens, 日期十位
- DU: Date Units, 日期个位

例如 2023/03/20 Monday:

$$Year = 23, \quad Month = 03, \quad Date = 20, \quad Weekday = 1 \text{ 或按芯片定义填写}$$

关键: RTC 的时间和日期不是普通二进制整数直接存进去, 而是以 **BCD** (Binary Coded Decimal, 二进制编码十进制) 形式分位存储。

Slides 7–8: BCD Format 与例子

BCD (Binary Coded Decimal, 二进制编码十进制) 的核心思想: 每一个十进制数字, 用 4 个二进制位表示。

BCD 与普通二进制的区别

普通二进制把整个数作为一个整体编码。例如十进制 29:

$$29_{10} = 11101_2$$

BCD 不这样做。BCD 先把 29 拆成两个十进制数字:

$$29 \Rightarrow 2 \text{ 和 } 9$$

然后每个数字用 4 bit:

$$2 = 0010, \quad 9 = 1001$$

所以:

$$29_{BCD} = 0010 \ 1001$$

十进制数	普通二进制	BCD 编码
9	1001	1001
12	1100	0001 0010
29	11101	0010 1001
59	111011	0101 1001

Slide 8 例子: 2023/03/20 Monday

如果要写入 RTC 日期, 关键拆法是:

- Year 23: 0010 0011
- Month 03: 0000 0011
- Date 20: 0010 0000
- Weekday Monday: 按手册约定写入对应编号。STM32 通常用 1 表示 Monday, 但考试或代码要以课件/头文件约定为准。

易错点: BCD 的 29 不是普通二进制 11101。RTC 里写 29 秒时要写成 0010 1001。你如果把普通二进制直接写进去, RTC 显示会错。

Slide 9: Steps to Set the Time for RTC

设置 RTC 时间的总步骤只有两类:

1. **Select proper clock source:** 选择合适的 RTC 时钟源。
2. **Set the initial time:** 写入初始时间和日期。

为什么先选 clock source?

RTC 本质上要靠时钟脉冲推进时间。如果没有时钟源, 或者时钟源频率不对, RTC 就不能正确走时。先给 RTC 一个稳定的节拍, 再写时间, 逻辑才完整。

第一性原理: 时间不是凭空产生的。RTC 必须把某个频率的时钟分频成 1 Hz, 才能每秒推进一次日历。

Slide 10: Clock Source

这一页讲 RTC 可以选择不同的时钟源。常见 RTC clock source 有:

- **LSE** (Low-Speed External clock, 低速外部晶振): 通常 32.768 kHz, 最适合 RTC。
- **LSI** (Low-Speed Internal clock, 低速内部 RC 时钟): 内部产生, 方便但精度较差。
- **HSE_RTC** (High-Speed External divided clock, 高速外部时钟分频后给 RTC): 来自外部高速晶振的分频版本。

为什么常用 32.768 kHz?

32.768 kHz = 32768 Hz = 2^{15} Hz。它非常适合用二进制分频得到 1 Hz:

$$32768 \div 32768 = 1$$

这就是手表晶振常用 32.768 kHz 的原因。

别机械背： LSE 不是因为名字高级才用，而是因为它低功耗、独立、频率适合分频到 1 Hz。

Slides 11–13: Clock Configuration - RTC 分频得到 1 Hz

RTC 需要把时钟源 RTCCLK 变成 1 Hz 的日历时钟。课件提到两个 prescaler:

- PREDIV_A: Asynchronous prescaler (异步预分频器)
- PREDIV_S: Synchronous prescaler (同步预分频器)

核心公式

$$ck_spre = \frac{RTCCLK}{(PREDIV_A + 1)(PREDIV_S + 1)}$$

目标通常是:

$$ck_spre = 1 \text{ Hz}$$

使用 LSE = 32768 Hz 的默认配置

课件说默认 RTC_PRER = 0x007F00FF, 对应:

$$PREDIV_A = 127, \quad PREDIV_S = 255$$

代入:

$$(PREDIV_A + 1)(PREDIV_S + 1) = 128 \times 256 = 32768$$

所以:

$$ck_spre = \frac{32768}{32768} = 1 \text{ Hz}$$

一句话： LSE 是 32768 Hz。用 127 和 255 两级分频后，刚好变成 1 Hz。RTC 日历单元就每秒更新一次。

Slides 14–21: RTC Initialization (8 Steps) 总览

这组页是本讲最像考试/实验的部分。它在讲 RTC 初始化的 8 个步骤。本质目标是：**解锁 RTC 写保护，进入初始化模式，配置分频与时间日期，再退出初始化模式，最后重新锁定。**

RTC 初始化大流程：

1. 写 RTC_WPR 解锁写保护。
2. 设置 RTC_ISR.INIT=1 进入初始化模式。
3. 等待 RTC_ISR.INITF=1，确认可以写时间日期。
4. 配置 RTC_PRER 分频器，通常 LSE 默认值就够。
5. 写 RTC_TR 和 RTC_DR 设置时间日期。
6. 配置 RTC_CR.FMT 选择 24 小时/12 小时格式。
7. 清除 RTC_ISR.INIT 退出初始化模式。
8. 写 RTC_WPR=0xFF 重新上锁。

下面逐步解释。

Slide 14: RTC Initialization Step 1 - 解锁 RTC_WPR

RTC_WPR 是 **RTC write protection register** (RTC 写保护寄存器)。RTC 很重要, 不能随便被程序误改, 所以默认有写保护。

要修改 RTC 配置, 必须先写入特定 key:

```
RTC->WPR = 0xCA;  
RTC->WPR = 0x53;
```

为什么是两次?

这是硬件设计的安全机制。只有按照固定顺序写入 0xCA 和 0x53, RTC 才认为你是真的要修改配置, 而不是程序跑飞误写。

直白理解: RTC_WPR 像“保险锁”。要改 RTC, 先输入两段密码: 0xCA、0x53。

Slides 15–16: RTC Initialization Steps 2–3 - INIT 与 INITF

这两页都在讲 RTC_ISR。RTC_ISR 是 **RTC initialization and status register** (RTC 初始化与状态寄存器)。

Step 2: 设置 INIT=1

INIT 是初始化模式控制位。写:

```
RTC->ISR |= RTC_ISR_INIT;
```

意思是:

请求 RTC 进入初始化模式, 暂停日历更新, 让软件可以安全修改时间日期和配置。

Step 3: 等待 INITF=1

INITF 是初始化模式确认标志。程序要等待:

```
while ((RTC->ISR & RTC_ISR_INITF) == 0) {  
    ; // wait  
}
```

意思是:

硬件已经准备好, 允许你写 RTC_TR、RTC_DR、RTC_PRER 等寄存器。

易错点: 设置 INIT 后不能马上无脑写。要等 INITF 变成 1。INIT 是你的请求, INITF 是硬件的确认。请求不等于完成。

Slide 17: RTC Initialization Step 4 - 配置 RTC_PRER

RTC_PRER 是 **RTC prescaler register** (RTC 预分频寄存器), 里面有:

- PREDIV_A[6:0]: 异步预分频值
- PREDIV_S[14:0]: 同步预分频值

如果时钟源选择 LSE = 32768 Hz, 课件说默认值已经合适:

$$RTC_PRER = 0x007F00FF$$

即:

$$PREDIV_A = 127, \quad PREDIV_S = 255$$

刚好得到：

$$\frac{32768}{(127+1)(255+1)} = 1 \text{ Hz}$$

实验思维：如果你用 LSE，很多情况下不用改 PRER。若时钟源换成 LSI 或 HSE_RTC，就必须重新计算分频值，否则 RTC 走时会快/慢。

Slide 18: RTC Initialization Step 5 - 写 RTC_TR 与 RTC_DR

这一步是真正写入初始时间和日期。

写 RTC_TR：时间

RTC_TR 包含小时、分钟、秒的 BCD 字段。例如设置 13:59:30，逻辑是：

- Hour = 13: HT=1, HU=3
- Minute = 59: MNT=5, MNU=9
- Second = 30: ST=3, SU=0

对应十六进制常见写法可以类似：

```
RTC->TR = 0x00135930; // 13:59:30 in BCD fields
```

这里不是普通整数 135930，而是把 BCD 字段放进对应 bit 位置。

写 RTC_DR：日期

RTC_DR 包含年、月、日期、星期。例如 2023/03/20 Monday：

- Year = 23
- Month = 03
- Date = 20
- Weekday = Monday

关键警告：TR 和 DR 的字段是 BCD。你不是把“秒数”或“日期总数”写进去，而是把每个十进制位拆开放到指定 bit field。

Slide 19: RTC Initialization Step 6 - RTC_CR 的 FMT 位

RTC_CR 是 RTC control register (RTC 控制寄存器)。这一页重点是 FMT。

FMT 是什么？

FMT 控制时间格式：

- FMT=0: 24-hour format, 24 小时制。
- FMT=1: 12-hour format, 12 小时制，需要配合 AM/PM 位。

课件说默认 FMT=0，也就是 24 小时制。

建议：初学和实验尽量用 24 小时制。12 小时制还要处理 PM 位，增加错误空间。工程里不缺这种自找麻烦的机会，不必主动制造。

Slide 20: RTC Initialization Step 7 - 退出初始化模式

Step 7 回到 RTC_ISR, 要清除 INIT:

```
RTC->ISR &= ~RTC_ISR_INIT;
```

含义:

软件告诉 RTC: 时间日期已经设置完了, 可以退出初始化模式, 恢复正常计时。

如果一直保持 INIT=1, RTC 可能不会正常更新时间。因此设置完必须退出。

易错点: 初始化不是永久状态。进入 INIT 是为了配置, 配置完要退出。否则你等着 RTC 走, 它可能根本没按你想的走。

Slide 21: RTC Initialization Step 8 - 重新锁定 RTC_WPR

最后写:

```
RTC->WPR = 0xFF;
```

这会重新启用写保护, 防止后续程序误写 RTC 寄存器。

完整闭环: 0xCA、0x53 是解锁; 0xFF 是锁回去。不要只会开锁不会关门。

Slides 22–23: Read From RTC - 读取 RTC 时间

这两页讲从 RTC 读取时间, 主要读 RTC_TR。

读取 RTC_TR

RTC_TR 存的是 BCD 字段。读出来后, 需要把字段取出, 再转换成人能看懂的十进制。

例如:

```
uint32_t tr = RTC->TR;
uint32_t sec_units = tr & 0xF;
uint32_t sec_tens = (tr >> 4) & 0x7;
uint32_t seconds = sec_tens * 10 + sec_units;
```

为什么不能直接把 RTC_TR 当普通数字?

如果 RTC_TR = 0x00135930, 它表示 13:59:30。你不能把 0x00135930 当成普通十进制数来使用。它是字段集合。

易错点: RTC 寄存器读出来是“打包字段”, 不是一个普通时间变量。要先 mask 和 shift, 再把 BCD 转成 binary/decimal。

Slides 24–25: Alarm 概念

Alarm (闹钟) 是 RTC 的条件触发功能。

RTC 正常每秒更新时间; Alarm 则检查当前时间是否匹配你设置的目标时间。如果匹配, 就可以:

- 设置 alarm flag。
- 产生 interrupt。
- 唤醒低功耗状态下的处理器。

直白理解: RTC 是手表; Alarm 是闹钟规则。手表一直走, 闹钟只在条件满足时响。

Slide 26: RTC Alarms - Alarm A 与 Alarm B

STM32 RTC 内部有两个类似的 alarm:

- Alarm A (闹钟 A)
- Alarm B (闹钟 B)

这一页重点是 RTC_ALRMAR: **RTC alarm A register** (RTC 闹钟 A 寄存器)。

Alarm register 存什么?

RTC_ALRMAR 存的是闹钟匹配条件, 包括:

- 秒字段
- 分字段
- 小时字段
- 日期或星期字段
- mask 位: 决定某些字段是否参与比较

Mask 位的意义

Mask (掩码) 的含义是: **忽略某个字段, 不让它参与比较。**

例如:

- 如果日期字段 mask 掉, 那么每天这个时间都会触发。
- 如果日期/星期不 mask, 那么只有指定日期或星期才触发。

Alarm 本质: 当前 RTC 时间和 Alarm register 里未被 mask 的字段逐项比较; 全部匹配, 就触发 alarm。

Slide 27: Set Alarm - Every Monday vs Every Day

这一页给出两个需求:

- Every Monday, 04:50:00 pm: 每周一 16:50:00 触发。
- Every Day, 04:50:00 pm: 每天 16:50:00 触发。

每周一 16:50:00

要让星期参与比较:

- 小时 = 16 或 12 小时制下 04 PM
- 分钟 = 50
- 秒 = 00
- 星期 = Monday
- 不忽略星期字段

每天 16:50:00

要忽略日期/星期字段:

- 小时、分钟、秒参与比较。
- 日期/星期 mask 掉。

区别一句话: Every Monday 要比较“星期”; Every Day 不比较“星期/日期”。这靠 Alarm mask 位控制。

Slide 28: Steps to Configure ALARM A

配置 Alarm A 的逻辑步骤可以概括为：

1. 禁用 Alarm A，防止配置过程中误触发。
2. 等待 Alarm A write flag，确认可以写 alarm register。
3. 写 RTC_ALRMAR，设置时间字段和 mask 字段。
4. 清除旧的 alarm flag。
5. 使能 Alarm A。
6. 如果要中断，还要使能 RTC Alarm interrupt 和 EXTI18/NVIC 相关路径。

工程原则：改一个正在运行的 alarm 前先 disable，写完再 enable。边跑边改寄存器是制造玄学 bug 的常见方式。

Slides 29–30: Alarm Interrupts - RTC_CR 与 RTC_ISR

这两页进入 Alarm interrupt。

RTC_CR 中的关键位

- ALRAE: Alarm A enable, 启用 Alarm A 本身。
- ALRAIE: Alarm A interrupt enable, 允许 Alarm A 产生中断。
- ALRBE: Alarm B enable。
- ALRBIE: Alarm B interrupt enable。

RTC_ISR 中的关键位

- ALRAF: Alarm A flag, 表示 Alarm A 已触发。
- ALRBF: Alarm B flag, 表示 Alarm B 已触发。
- ALRAWF: Alarm A write flag, 表示 Alarm A register 可以写。
- ALRBWF: Alarm B write flag, 表示 Alarm B register 可以写。

不要混淆：ALRAE 是“闹钟 A 功能打开”；ALRAIE 是“闹钟 A 触发后允许产生中断”。一个是 alarm 功能，一个是 interrupt 输出。

Slides 31–32: RTC Alarm Interrupt 连接到 EXTI18

这两页很关键：RTC Alarm interrupt 不是直接飞进 CPU，而是内部连接到 **EXTI18**（External Interrupt Line 18，外部/扩展中断线 18）。

完整链路：

1. RTC 时间达到 Alarm A 条件。
2. RTC 设置 ALRAF=1。
3. RTC Alarm 信号连接到 EXTI line 18。
4. EXTI18 pending 被置位。
5. NVIC 看到 RTC alarm 对应中断请求。
6. CPU 进入 RTC Alarm ISR。

为什么 RTC 还要走 EXTI?

EXTI 不只处理 GPIO 引脚，它也可以处理某些内部事件。RTC Alarm 是内部事件，但它被接到 EXTI18 这条线，由 EXTI/NVIC 中断系统统一管理。

易错点：EXTI 不等于“外部引脚专用”。很多内部外设事件也会接到 EXTI line，例如 RTC Alarm 接 EXTI18。

Slides 33–42: EXTI 寄存器与 RTC Alarm 相关配置

这组页列出 EXTI 的若干寄存器。虽然图很多，但核心只需要抓住每个寄存器在中断链路中的角色。

1. EXTI_RTSR1 / EXTI_RTSR2

Rising trigger selection register（上升沿触发选择寄存器）。

作用：选择某条 EXTI line 是否在 rising edge 上触发。

对于 RTC Alarm 这种内部事件，通常需要按手册要求配置 EXTI18 的触发方式。

2. EXTI_FTSR1 / EXTI_FTSR2

Falling trigger selection register（下降沿触发选择寄存器）。

作用：选择某条 EXTI line 是否在 falling edge 上触发。GPIO 按键常用上升沿或下降沿；RTC Alarm 通常关注其内部事件触发路径。

3. EXTI_IMR1 / EXTI_IMR2

Interrupt mask register（中断屏蔽寄存器）。

作用：决定某条 EXTI line 是否允许产生中断请求。

如果 IMR 对应 bit = 0，即使事件来了，也不会送到 NVIC 形成 interrupt。

4. EXTI_PR1 / EXTI_PR2

Pending register（挂起寄存器）。

作用：记录某条 EXTI line 的事件已经发生。

ISR 中通常要清除 pending bit，否则中断可能反复进入。

5. EXTI_SWIER1 / EXTI_SWIER2

Software interrupt event register（软件中断事件寄存器）。

作用：软件主动制造一个 EXTI 事件，用于测试或软件触发中断。

寄存器	中文	直白作用
RTSR	上升沿触发选择	什么上升沿会触发
FTSR	下降沿触发选择	什么下降沿会触发
IMR	中断屏蔽	允许/禁止送中断
PR	挂起标志	事件是否已经发生
SWIER	软件事件	软件模拟触发

最常见 bug：RTC Alarm flag 清了，但 EXTI pending 没清；或者 EXTI pending 清了，但 RTC ALRAF 没清。两个层级都要检查。

Slide 43: RTC Alarm Interrupt Handler - 在哪里检查代码

最后一页提醒：RTC Alarm interrupt handler 中要检查你的代码。意思是：当 RTC Alarm 触发后，CPU 会进入对应 ISR，你需要在 ISR 中完成两类动作。

ISR 中应该做什么？

1. 判断是不是 Alarm A 或 Alarm B 触发。
2. 执行你的任务，例如点亮 LED、更新时间变量、唤醒主循环。
3. 清除 RTC alarm flag，例如 ALRAF。
4. 清除 EXTI18 pending flag。

伪代码结构

```
void RTC_Alarm_IRQHandler(void)
{
    if (RTC->ISR & RTC_ISR_ALRAF) {
        // 1. User action
        // e.g., toggle LED, set a global flag, wake main loop

        // 2. Clear RTC Alarm A flag
        RTC->ISR &= ~RTC_ISR_ALRAF;

        // 3. Clear EXTI18 pending flag
        EXTI->PR1 = (1UL << 18);
    }
}
```

注意：不同 STM32 库/头文件对清 flag 的写法可能有差异。考试理解链路即可；实验代码必须按课程模板和芯片参考手册写。

Lecture 16 总结：你必须掌握的内容

1. RTC 是什么

RTC 是独立低功耗日历时钟，能记录时间和日期，适合低功耗计时、闹钟唤醒、时间戳。

2. RTC 为什么要分频

RTC 时钟源通常是 LSE 32768 Hz，要通过 PREDIV_A 和 PREDIV_S 分频成 1 Hz，日历才每秒更新。

3. BCD 是什么

BCD 是每个十进制数字用 4 bit 存储。RTC 的时间日期字段使用 BCD，不能当普通二进制数直接理解。

4. RTC 初始化核心流程

```
Unlock WPR -> INIT=1 -> wait INITF -> set PRER/TR/DR/CR
-> INIT=0 -> lock WPR
```

5. Alarm 的本质

Alarm 是匹配规则。当前时间和 alarm register 中未被 mask 的字段匹配时，触发 Alarm flag 和 interrupt。

6. RTC Alarm 中断链路

```
RTC Alarm match
-> ALRAF set
-> EXTI18 pending
-> NVIC request
-> CPU enters RTC Alarm IRQHandler
-> clear RTC flag and EXTI pending
```

最终一句话： Lecture 16 的核心不是“RTC 有很多寄存器”，而是：RTC 用稳定低功耗时钟分频出 1 Hz，以 BCD 格式维护日历时间，并通过 Alarm + EXTI18 + NVIC 这条链路触发中断。